

RAG con Chroma sobre el Agente Personal de Planificación Académica

Objetivo

Implementar un sistema **RAG** (*Retrieval-Augmented Generation*, generación aumentada por recuperación) en Python con **Chroma** que resuelva un caso personal real, integrándolo al agente ReAct de planificación académica construido en la Sesión 12 (`Tarea_Agente_Personal`).

Caso personal: de búsqueda literal a búsqueda semántica real

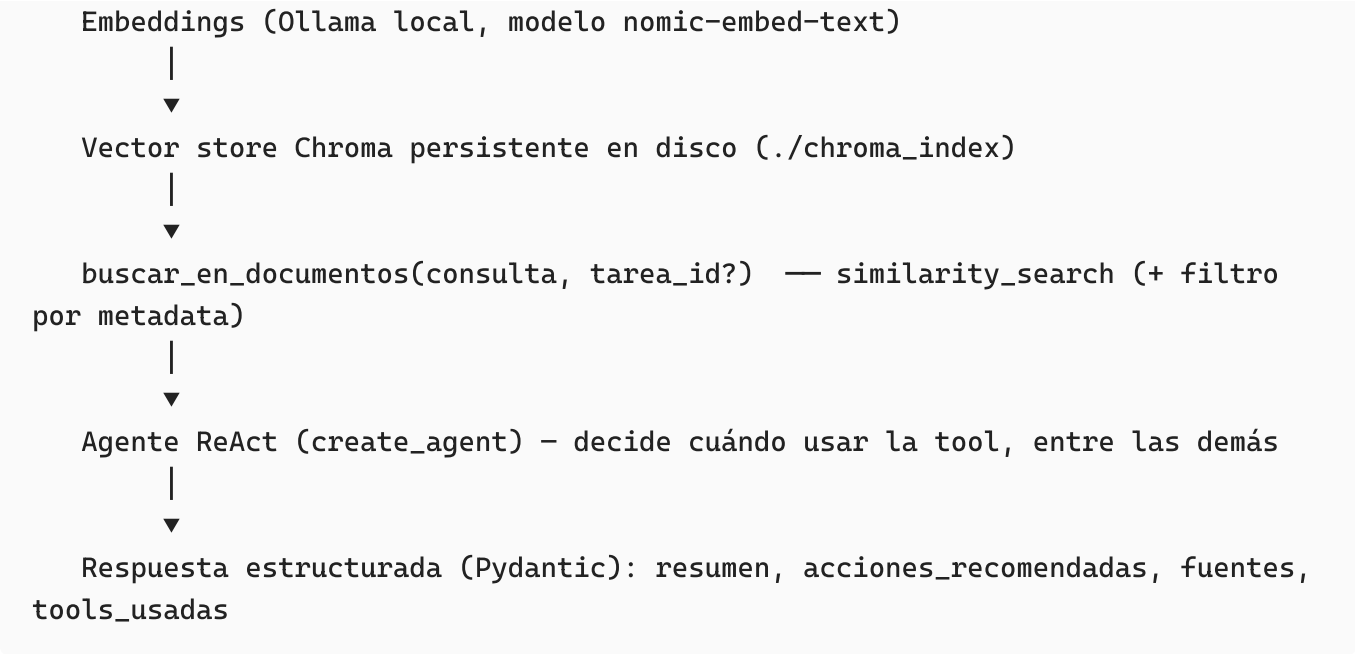
El agente de la Sesión 12 ya podía leer los documentos de materiales de cada tarea (`materiales/<carpeta>/`), pero su herramienta `buscar_en_documentos` hacía **coincidencia literal de texto** — comparaba línea por línea si la consulta aparecía tal cual (sin tildes) dentro del documento. Eso falla en un caso muy común: preguntar algo con **otras palabras** que las del documento. Por ejemplo, si `instrucciones.txt` dice *"el entregable es un Google Doc con descripción y código"*, preguntar *"¿qué tengo que enviar?"* no encontraba nada, porque ninguna de esas palabras aparece literalmente en el texto.

Ese es el problema personal que este RAG resuelve: reemplazar la búsqueda literal por una **búsqueda semántica real** — el agente entiende el *significado* de la pregunta, no solo sus palabras exactas, y responde citando de qué archivo salió cada dato. Como respaldo, cuando la pregunta es general y no depende de mis documentos (o `buscar_en_documentos` no encuentra nada útil), el agente puede complementar con una búsqueda web real (Tavily).

Arquitectura RAG

```

materiales/<tarea>/*. {pdf,docx,txt,md,py}
    | (extracción de texto: pypdf / python-docx / texto plano)
    ▼
    Chunking (RecursiveCharacterTextSplitter, 800 caracteres, solape 120)
    |
    ▼
```



Cada chunk se indexa con metadata `{"archivo": <nombre>, "ruta_contexto": <carpeta relativa>}` — el mismo formato que usa `ruta_contexto` en `tareas.json` — para poder filtrar la búsqueda a los materiales de una tarea específica cuando se da `tarea_id` , o buscar en todo lo indexado si no.

Por qué Ollama local para embeddings: evita depender de una API de pago (no tengo `OPENAI_API_KEY`) y reutiliza infraestructura que ya tenía corriendo localmente para otro módulo del programa — sin instalar nada nuevo salvo el modelo de embeddings (`ollama pull nomic-embed-text` , ~274 MB, una sola vez).

Herramientas del agente

Herramienta	Función
<code>consultar_tareas</code>	Lista las tareas registradas y su estado.
<code>agregar_tarea</code>	Registra una tarea nueva.
<code>calcular_prioridad</code>	Calcula el puntaje de urgencia de una tarea.
<code>generar_plan</code>	Arma y guarda el plan calculado desde <code>tareas.json</code> .
<code>actualizar_estado</code>	Cambia el estado de una tarea (pendiente/iniciada/completada).
<code>guardar_plan_detallado</code>	Guarda un plan de texto libre ya redactado en el chat.
<code>inspeccionar_carpeta</code>	Lista los documentos disponibles de una tarea.

Herramienta	Función
<code>buscar_en_documentos</code>	RAG real: búsqueda semántica con Chroma sobre los materiales, opcionalmente filtrada por tarea. Reemplaza la búsqueda literal de la Sesión 12.
<code>buscar_en_la_web</code>	Búsqueda web (Tavily) como respaldo cuando los materiales locales no alcanzan.

La salida de cada turno se fuerza a un formato estructurado (`RespuestaAgente` : `resumen` , `acciones_recomendadas` , `fuentes` , `tools_usadas`) para que quede explícito qué herramientas se usaron y de qué documentos salió cada dato — trazabilidad directa del RAG.

Ejemplo de uso

Tú: ¿qué tengo que entregar en la tarea del agente ReAct?

Acción: `buscar_en_documentos("qué debo entregar", tarea_id=1)`
→ `[instrucciones.txt]` "...el entregable es un Google Doc con descripción y código..."

Respuesta: `resumen` con el entregable citando `instrucciones.txt` como fuente, aunque la pregunta no usó las palabras exactas del documento.

Verificación real (no solo revisión de código)

Se probó el pipeline completo, no solo se asumió que funcionaría:

1. **RAG aislado** (`rag.buscar(...)` , sin el LLM de por medio): una consulta parafraseada que *no* usa las palabras exactas del documento ("¿qué debo mandarle al profesor al final?") recuperó igual el fragmento correcto de `instrucciones.txt` — confirma que es búsqueda semántica real, no coincidencia de texto. El filtro por `ruta_contexto` también se probó: sobre una carpeta vacía (`Investigacion`) devuelve una lista vacía sin error, y sobre la carpeta correcta recupera el fragmento esperado.
2. **Agente completo** (con Claude vía Anthropic, `AGENT_MODEL=anthropic`): en la primera corrida el modelo respondió con un mensaje tipo *"dame un momento mientras consulto tus tareas y materiales"* como respuesta final estructurada — sin haber llamado ninguna herramienta (`fuentes` y `tools_usadas` vacíos). Es un caso real de que la salida estructurada por sí sola no obliga al modelo a usar las tools antes de responder. Se corrigió agregando una regla

explícita al `PROMPT_SISTEMA` prohibiendo respuestas de "espera"/"estoy consultando" como respuesta final. Tras el fix, la misma pregunta sí dispara `consultar_tareas` → `inspeccionar_carpeta` → `buscar_en_documentos`, y la respuesta final cita `instrucciones.txt` en `fuentes` con el contenido real del documento.

Código completo

`rag.py` — índice vectorial (Chroma)

```
"""
Índice vectorial (RAG) sobre los materiales del agente personal, con Chroma.

Recorre materiales/ (todas las subcarpetas de tareas), extrae texto de cada
documento compatible (.pdf, .docx, .txt, .md, .py), lo divide en chunks y lo
indexa en una coleccion Chroma persistente en disco. El resto del proyecto
(agente_planificacion_rag.py) solo llama a buscar() para hacer busqueda
semantica sobre ese indice.

Embeddings: Ollama local ('nomic-embed-text', mismo servidor Ollama que ya
corre en esta maquina para otros modulos) -- se instala una sola vez con:
    ollama pull nomic-embed-text

Reconstruir el indice a mano (por ejemplo tras agregar documentos nuevos):
    python rag.py --reindexar
"""

import os
import shutil
import sys
from pathlib import Path

from langchain_chroma import Chroma
from langchain_core.documents import Document
from langchain_ollama import OllamaEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter

CARPETA_MATERIALES = Path(__file__).parent / "materiales"
CARPETA_INDICE = Path(__file__).parent / "chroma_index"
NOMBRE_COLECCION = "materiales_agente_personal"
EXTENSIONES_ADMITIDAS = {".pdf", ".docx", ".txt", ".md", ".py"}
MODELO_EMBEDDINGS = os.getenv("OLLAMA_EMBEDDINGS_MODEL", "nomic-embed-text")
OLLAMA_BASE_URL = os.getenv("OLLAMA_BASE_URL", "http://localhost:11434")

_embeddings: OllamaEmbeddings | None = None
```

```

_vector_store: Chroma | None = None

def _obtener_embeddings() -> OllamaEmbeddings:
    global _embeddings
    if _embeddings is None:
        _embeddings = OllamaEmbeddings(model=MODELO_EMBEDDINGS,
base_url=OLLAMA_BASE_URL)
    return _embeddings

def _extraer_texto_archivo(ruta: Path) -> str:
    sufijo = ruta.suffix.lower()
    if sufijo in {".txt", ".md", ".py"}:
        return ruta.read_text(encoding="utf-8", errors="ignore")
    if sufijo == ".docx":
        from docx import Document as DocumentoWord
        return "\n".join(p.text for p in DocumentoWord(ruta).paragraphs)
    if sufijo == ".pdf":
        from pypdf import PdfReader
        return "\n".join(pagina.extract_text() or "" for pagina in
PdfReader(ruta).pages)
    return ""

def _cargar_documentos() -> tuple[list[Document], list[str]]:
    """Recorre materiales/ y arma un Document (LangChain) por chunk de cada
archivo compatible."""
    splitter = RecursiveCharacterTextSplitter(chunk_size=800,
chunk_overlap=120)
    documentos: list[Document] = []
    ids: list[str] = []

    for archivo in sorted(CARPETA_MATERIALES.rglob("*")):
        if not archivo.is_file() or archivo.suffix.lower() not in
EXTENSIONES_ADMITIDAS:
            continue
        try:
            texto = _extraer_texto_archivo(archivo)
        except Exception as error:
            print(f"[rag] No se pudo leer {archivo.name}: {error}")
            continue
        if not texto.strip():
            continue

    # Mismo formato que ruta_contexto en tareas.json (ej.

```

```

"materiales/TareaReAct"),
    # para poder filtrar la búsqueda por la carpeta de una tarea
    # específica.
    ruta_contexto =
str(archivo.parent.relative_to(CARPETA_MATERIALES.parent)).replace("\\", "/")

    for i, chunk in enumerate(splitter.split_text(texto)):
        documentos.append(Document(
            page_content=chunk,
            metadata={"archivo": archivo.name, "ruta_contexto":
ruta_contexto},
        ))
        ids.append(f"{archivo.relative_to(CARPETA_MATERIALES)}::{
{i}}".replace("\\", "/"))

    return documentos, ids

def construir_o_cargar_indice(forzar_reindexado: bool = False) -> Chroma:
    """Crea el índice si no existe (o si se fuerza), y lo deja listo para
    buscar()."""
    global _vector_store

    if forzar_reindexado and CARPETA_INDICE.exists():
        shutil.rmtree(CARPETA_INDICE)

    indexar = forzar_reindexado or not CARPETA_INDICE.exists()

    vector_store = Chroma(
        collection_name=NOMBRE_COLECCION,
        persist_directory=str(CARPETA_INDICE),
        embedding_function=_obtener_embeddings(),
    )

    if indexar:
        documentos, ids = _cargar_documentos()
        if documentos:
            vector_store.add_documents(documents=documentos, ids=ids)
            print(f"[rag] Índice construido con {len(documentos)} fragmento(s)
de materiales/.")
        else:
            print("[rag] No se encontraron documentos compatibles en
materiales/.")

    _vector_store = vector_store
    return _vector_store

```

```
def buscar(consulta: str, k: int = 4, ruta_contexto: str | None = None) ->
list[Document]:
    """Busqueda semantica sobre el indice. Si ruta_contexto se da, filtra por
    esa carpeta."""
    if _vector_store is None:
        construir_o_cargar_indice()
    filtro = {"ruta_contexto": ruta_contexto} if ruta_contexto else None
    return _vector_store.similarity_search(consulta, k=k, filter=filtro)

if __name__ == "__main__":
    construir_o_cargar_indice(forzar_reindexado="--reindexar" in sys.argv)
```

agente_planificacion_rag.py — agente ReAct con la tool RAG integrada

```
"""
Agente personal de planificacion academica, construido con LangChain
(create_agent) siguiendo el patron ReAct: prioriza tareas pendientes segun
fecha limite y prioridad, arma un plan de tiempo, y responde preguntas sobre
los documentos asociados a cada tarea usando RAG real (Chroma + embeddings),
en vez de una busqueda de texto literal.

Version RAG (Sesion 13) de Tarea_Agente_Personal (Sesion 12): mismo agente
de planificacion, con buscar_en_documentos reemplazada por busqueda semantica
(ver rag.py), y una tool nueva de busqueda web (Tavily) como respaldo cuando
los materiales locales no alcanzan.

Backends soportados via AGENT_MODEL (.env):
    AGENT_MODEL=anthropic      -> Claude via API de Anthropic
    AGENT_MODEL=gemma-lmstudio -> Gemma 4 E4B via LM Studio (local)

Antes de correr el chat, asegurate de tener las dependencias instaladas en
el venv (ver requirements.txt). Para correr el chat:
    python agente_planificacion_rag.py
    python agente_planificacion_rag.py --reindexar # fuerza reconstruir el
    indice RAG
"""

import json
import os
import sys
from datetime import date, datetime
```

```

from pathlib import Path

from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.tools import tool
from pydantic import BaseModel, Field

import rag

load_dotenv()

RUTA_TAREAS = Path(__file__).parent / "tareas.json"
CARPETA_PLANES = Path(__file__).parent / "planes_generados"

# Nada se lee fuera de esta carpeta. Se puede apuntar a otra ubicacion
# con la variable de entorno CARPETA_AUTORIZADA.
CARPETA_AUTORIZADA = Path(
    os.getenv("CARPETA_AUTORIZADA", str(Path(__file__).parent / "materiales"))
).resolve()

EXTENSIONES_ADMITIDAS = {".pdf", ".docx", ".txt", ".md", ".py"}
PESO_PRIORIDAD = {"alta": 3, "media": 2, "baja": 1}
TAVILY_API_KEY = os.getenv("TAVILY_API_KEY")

def _cargar_tareas() -> list[dict]:
    if not RUTA_TAREAS.exists():
        return []
    return json.loads(RUTA_TAREAS.read_text(encoding="utf-8"))

def _guardar_tareas(tareas: list[dict]) -> None:
    RUTA_TAREAS.write_text(json.dumps(tareas, ensure_ascii=False, indent=2),
encoding="utf-8")

def _buscar_tarea(tarea_id: int) -> dict | None:
    return next((t for t in _cargar_tareas() if t["id"] == tarea_id), None)

def _validar_ruta(ruta: str) -> Path | None:
    ruta_resuelta = (Path(__file__).parent / ruta).resolve() if not
Path(ruta).is_absolute() else Path(ruta).resolve()
    if ruta_resuelta == CARPETA_AUTORIZADA or CARPETA_AUTORIZADA in
ruta_resuelta.parents:
        return ruta_resuelta

```



```
return None
```

```
def _puntaje_urgencia(tarea: dict) -> float:
    # peso * 100 / (dias restantes + 1): a mayor prioridad y menos tiempo
    # restante, mayor el puntaje.
    dias_restantes = max((date.fromisoformat(tarea["fecha_limite"]) -
date.today()).days, 0)
    peso = PESO_PRIORIDAD.get(tarea["prioridad"], 1)
    return peso * 100 / (dias_restantes + 1)
```

```
@tool
```

```
def consultar_tareas() -> str:
    """Devuelve todas las tareas registradas, con su estado actual."""
    tareas = _cargar_tareas()
    if not tareas:
        return "No hay tareas registradas."
    return "\n".join(
        f"[{t['id']}] {t['nombre']} - curso: {t['curso']} - vence:
{t['fecha_limite']} "
        f"- prioridad: {t['prioridad']} - estado: {t['estado']}"
        for t in tareas
    )
```

```
@tool
```

```
def agregar_tarea(
    nombre: str, curso: str, fecha_limite: str, duracion_minutos: int,
    prioridad: str, ruta_contexto: str = "", entregable: str = "",
) -> str:
    """
    Registra una tarea nueva. fecha_limite en formato YYYY-MM-DD,
    prioridad 'alta', 'media' o 'baja'. Si falta algun dato pidesele al
    usuario en vez de inventarlo.
    """
    tareas = _cargar_tareas()
    nuevo_id = max((t["id"] for t in tareas), default=0) + 1
    tareas.append({
        "id": nuevo_id, "nombre": nombre, "curso": curso, "fecha_limite":
fecha_limite,
        "duracion_estimada_minutos": duracion_minutos, "prioridad": prioridad,
        "estado": "pendiente", "ruta_contexto": ruta_contexto, "entregable":
entregable,
    })
    _guardar_tareas(tareas)
```

```
return f"Tarea '{nombre}' registrada con id {nuevo_id}."
```

```
@tool
```

```
def calcular_prioridad(tarea_id: int) -> str:
```

```
    """Calcula el puntaje de urgencia de una tarea. Usala antes de decidir  
    cual atender primero."""
```

```
    tarea = _buscar_tarea(tarea_id)
```

```
    if tarea is None:
```

```
        return f"No existe una tarea con id {tarea_id}."
```

```
    dias_restantes = max((date.fromisoformat(tarea["fecha_limite"]) -  
date.today()).days, 0)
```

```
    return (
```

```
        f"Tarea {tarea_id} ({tarea['nombre']}): puntaje de urgencia "
```

```
        f"{_puntaje_urgencia(tarea):.1f} (prioridad={tarea['prioridad']},  
vence en {dias_restantes} dia(s))."
```

```
    )
```

```
@tool
```

```
def generar_plan(minutos_disponibles: int) -> str:
```

```
    """
```

```
    Arma un plan con las tareas pendientes, de mayor a menor urgencia,  
    con 10 minutos de descanso entre cada una, y lo guarda como .md en  
    planes_generados/. Usala despues de calcular_prioridad.
```

```
    """
```

```
    pendientes = sorted(
```

```
        (t for t in _cargar_tareas() if t["estado"] != "completada"),
```

```
        key=_puntaje_urgencia, reverse=True,
```

```
    )
```

```
    filas: list[str] = []
```

```
    minutos_usados = 0
```

```
    for tarea in pendientes:
```

```
        duracion = tarea["duracion_estimada_minutos"]
```

```
        if minutos_usados + duracion > minutos_disponibles:
```

```
            continue
```

```
        filas.append(f"| {tarea['nombre']} | {duracion} min |  
{tarea['prioridad']} |")
```

```
        minutos_usados += duracion
```

```
        if minutos_usados < minutos_disponibles:
```

```
            minutos_usados += 10
```

```
    if not filas:
```

```
        return "No hay tiempo suficiente para ninguna tarea pendiente con el  
tiempo disponible."
```

```

fecha_generacion = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
codigo_plan = f"PLAN-{datetime.now().strftime('%Y%m%d%H%M%S')}"
contenido_md = f"""\# Plan academico

**Código:** {codigo_plan}
**Fecha de generación:** {fecha_generacion}
**Tiempo disponible:** {minutos_disponibles} min (usados: {minutos_usados}
min, incluye descansos de 10 min entre tareas)

## Bloques

| Tarea | Duración | Prioridad |
|---|---|---|
{chr(10)}.join(filas)}
"""

    try:
        CARPETA_PLANES.mkdir(exist_ok=True)
        archivo_plan = CARPETA_PLANES / f"{codigo_plan}.md"
        archivo_plan.write_text(contenido_md, encoding="utf-8")
    except OSError as error:
        return f"No se pudo guardar el plan: {error}"

    return (
        f"Plan {codigo_plan} generado con {len(filas)} tarea(s), "
        f"{minutos_usados} de {minutos_disponibles} minutos usados. "
        f"Reporte guardado en {archivo_plan}."
    )

@tool
def actualizar_estado(tarea_id: int, nuevo_estado: str) -> str:
    """Cambia el estado de una tarea a 'pendiente', 'iniciada' o
    'completada'."""
    if nuevo_estado not in {"pendiente", "iniciada", "completada"}:
        return f"Estado '{nuevo_estado}' invalido. Usa: pendiente, iniciada o
completada."
    tareas = _cargar_tareas()
    for t in tareas:
        if t["id"] == tarea_id:
            t["estado"] = nuevo_estado
            _guardar_tareas(tareas)
            return f"Tarea {tarea_id} actualizada a estado '{nuevo_estado}'."
    return f"No existe una tarea con id {tarea_id}."

```

@tool

```
def guardar_plan_detallado(tarea_id: int, titulo: str, contenido_markdown:
str) -> str:
    """
    Guarda en planes_generados/ un plan que ya escribiste en el chat
    (por ejemplo un desglose por fases), tal cual, sin recalcular nada.
    Usala cuando el usuario pida guardar o exportar algo que ya
    redactaste – generar_plan no sirve para esto porque arma su propia
    tabla desde tareas.json.
    """
    tarea = _buscar_tarea(tarea_id)
    if tarea is None:
        return f"No existe una tarea con id {tarea_id}."

    fecha_generacion = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    codigo_plan = f"PLAN-DETALLADO-{datetime.now().strftime('%Y%m%d%H%M%S')}"
    contenido_final = (
        f"# {titulo}\n\n**Código:** {codigo_plan}\n"
        f"**Fecha de generación:** {fecha_generacion}\n**Tarea:**
{tarea['nombre']}\n\n"
        f"---\n\n{contenido_markdown}"
    )

    try:
        CARPETA_PLANES.mkdir(exist_ok=True)
        archivo_plan = CARPETA_PLANES / f"{codigo_plan}.md"
        archivo_plan.write_text(contenido_final, encoding="utf-8")
    except OSError as error:
        return f"No se pudo guardar el plan: {error}"

    return f"Plan detallado guardado en {archivo_plan}."
```

@tool

```
def inspeccionar_carpeta(tarea_id: int) -> str:
    """
    Lista los archivos compatibles (PDF, DOCX, TXT, Markdown, Python) en
    la carpeta de materiales de una tarea, a partir de su id.
    """
    tarea = _buscar_tarea(tarea_id)
    if tarea is None:
        return f"No existe una tarea con id {tarea_id}."
    if not tarea.get("ruta_contexto"):
        return f"La tarea {tarea_id} no tiene materiales asociados."
```

```

    carpeta = _validar_ruta(tarea["ruta_contexto"])
    if carpeta is None:
        return f"La carpeta de materiales de la tarea {tarea_id} esta fuera de la carpeta autorizada."
    if not carpeta.is_dir():
        return f"La carpeta '{tarea['ruta_contexto']}' de la tarea {tarea_id} no existe."

    archivos = sorted(f.name for f in carpeta.iterdir() if f.suffix.lower() in EXTENSIONES_ADMITIDAS)
    if not archivos:
        return f"No se encontraron documentos compatibles en la carpeta de la tarea {tarea_id}."
    return f"Documentos encontrados para la tarea {tarea_id}: " + ", ".join(archivos)

```

@tool

```

def buscar_en_documentos(consulta: str, tarea_id: int | None = None) -> str:
    """
    Busca semanticamente (RAG con Chroma) dentro de los documentos de materiales/. Si se da tarea_id, filtra por la carpeta de materiales de esa tarea; si no, busca en todo el material indexado. A diferencia de una busqueda de texto literal, encuentra fragmentos relevantes aunque la pregunta no use las mismas palabras que el documento.

    Args:
        consulta: Pregunta o tema a buscar.
        tarea_id: Id de la tarea para limitar la busqueda a su carpeta de materiales (opcional; si se omite, busca en todos los materiales).
    """
    ruta_contexto = None
    if tarea_id is not None:
        tarea = _buscar_tarea(tarea_id)
        if tarea is None:
            return f"No existe una tarea con id {tarea_id}."
        if not tarea.get("ruta_contexto"):
            return f"La tarea {tarea_id} no tiene materiales asociados."
        ruta_contexto = tarea["ruta_contexto"]

    resultados = rag.buscar(consulta, k=4, ruta_contexto=ruta_contexto)
    if not resultados:
        return f"No se encontro informacion relevante para '{consulta}' en los materiales indexados."
    return "\n\n".join(f"[{doc.metadata['archivo']}] {doc.page_content}" for doc in resultados)

```

```

@tool
def buscar_en_la_web(consulta: str) -> str:
    """
    Busca informacion en la web con Tavily. Usala solo cuando
    buscar_en_documentos no encontro nada util en los materiales locales, o
    el usuario pregunta algo general que no depende de sus documentos (ej.
    conceptos, definiciones).
    """
    if not TAVILY_API_KEY:
        return "Busqueda web no disponible: falta configurar TAVILY_API_KEY en
el archivo .env."

    from tavily import TavilyClient
    client = TavilyClient(TAVILY_API_KEY)
    respuesta = client.search(query=consulta, search_depth="advanced")

    resultados = respuesta.get("results", [])
    if not resultados:
        return f"No se encontraron resultados en la web para '{consulta}'."

    return "\n\n".join(
        f"{r.get('title', 'Sin titulo')}\n{r.get('content', '')}\nFuente:
{r.get('url', '')}"
        for r in resultados
    )

class RespuestaAgente(BaseModel):
    resumen: str = Field(description="Resumen breve de la respuesta al
usuario")
    acciones_recomendadas: str = Field(description="Proximos pasos concretos
que el usuario deberia tomar")
    fuentes: list[str] = Field(default_factory=list, description="Archivos o
resultados web usados como referencia")
    tools_usadas: list[str] = Field(default_factory=list, description="Nombres
de las herramientas utilizadas para responder")

PROMPT_SISTEMA = """
Eres un agente personal de planificacion academica. Ayudas a priorizar
tareas pendientes y a responder preguntas sobre los materiales asociados a
cada una, usando siempre tus herramientas – nunca inventes fechas,
prioridades ni contenido de documentos que no hayas consultado.

```

Flujo recomendado:

1. Usa `consultar_tareas` para ver que hay pendiente.
2. Usa `calcular_prioridad` sobre las tareas relevantes para decidir cual atender primero (nunca decidas la urgencia "a ojo").
3. Si el usuario da un tiempo disponible, usa `generar_plan` para distribuir las tareas dentro de ese tiempo.
4. Si el usuario quiere empezar una tarea o pregunta algo sobre sus materiales, usa `inspeccionar_carpeta` para ver que documentos existen, y `buscar_en_documentos` (busqueda semantica con RAG, no busqueda literal) para responder preguntas de contenido – funciona aunque la pregunta no use las mismas palabras del documento.
5. Si `buscar_en_documentos` no encuentra nada util, o la pregunta es general y no depende de los materiales del usuario (definiciones, conceptos, informacion actualizada), usa `buscar_en_la_web` como respaldo.
6. Usa `actualizar_estado` cuando el usuario indique que empezo o termino una tarea, y `agregar_tarea` cuando mencione una actividad nueva que no este registrada.
7. Si el usuario pide "guardar", "exportar" o "escribir en un archivo" un plan detallado que ya redactaste en la conversacion (distinto de la tabla de `generar_plan`), usa `guardar_plan_detallado` pasandole ese mismo contenido. Nunca digas que un plan quedo guardado sin haber llamado a una herramienta que efectivamente lo guarde.

Reglas:

- Solo puedes inspeccionar o buscar dentro de carpetas dentro de la carpeta autorizada; si el usuario pide otra ruta, rechazala.
- Se breve y concreto: prioridad, plan de tiempo y proximos pasos.
- Toda respuesta debe llenar los campos de `RespuestaAgente`: `resumen`, `acciones_recomendadas`, `fuentes` (archivos o resultados web citados) y `tools_usadas` (las herramientas que efectivamente llamaste).
- NUNCA generes la respuesta final (`RespuestaAgente`) sin haber llamado antes a las herramientas necesarias para responder la pregunta. Esta prohibido responder cosas como "dame un momento", "estoy consultando" o "espera mientras reviso" como respuesta final – eso no es una respuesta, es una excusa. Si la pregunta requiere informacion de `tareas.json` o de los materiales, llama la herramienta correspondiente en este mismo turno y recién entonces genera `RespuestaAgente` con el resultado real.

"""

```
AGENT_MODEL = os.getenv("AGENT_MODEL", "anthropic").lower()
```

```
def resolver_modelo():  
    if AGENT_MODEL == "gemma-lmstudio":  
        from langchain_openai import ChatOpenAI
```

```

        return ChatOpenAI(
            model=os.getenv("LMSTUDIO_MODEL", "google/gemma-4-e4b"),
            base_url=os.getenv("LMSTUDIO_BASE_URL",
"http://172.30.32.1:8666/v1"),
            api_key="lm-studio",
            temperature=0.3,
        )
    if AGENT_MODEL == "anthropic":
        return f"anthropic:{os.getenv('ANTHROPIC_MODEL', 'claude-sonnet-4-6')}"
    raise ValueError(f"AGENT_MODEL desconocido: {AGENT_MODEL!r}. Opciones: anthropic, gemma-lmstudio")

def _nombre_modelo() -> str:
    if AGENT_MODEL == "gemma-lmstudio":
        return f"{os.getenv('LMSTUDIO_MODEL', 'google/gemma-4-e4b')} (LM Studio, local)"
    return f"{os.getenv('ANTHROPIC_MODEL', 'claude-sonnet-4-6')} (Anthropic, de pago)"

# Construye (o carga si ya existe) el indice RAG antes de crear el agente.
# `python agente_planificacion_rag.py --reindexar` fuerza reconstruirlo.
rag.construir_o_cargar_indice(forzar_reindexado="--reindexar" in sys.argv)

agent = create_agent(
    model=resolver_modelo(),
    system_prompt=PROMPT_SISTEMA,
    tools=[
        consultar_tareas, agregar_tarea, calcular_prioridad, generar_plan,
        actualizar_estado, guardar_plan_detallado, inspeccionar_carpeta,
        buscar_en_documentos, buscar_en_la_web,
    ],
    response_format=RespuestaAgente,
)

def iniciar_chat() -> None:
    print("=" * 60)
    print("AGENTE PERSONAL DE PLANIFICACION ACADEMICA (con RAG)")
    print("=" * 60)
    print(f"Modelo: {_nombre_modelo()}")
    print('Ejemplo: "¿Que tengo que entregar en la tarea del agente ReAct?"')
    print("Escribe 'salir' para finalizar.")
    print()

```



```

historial: list = []
while True:
    try:
        solicitud = input("Tu: ").strip()
        if not solicitud:
            continue
        if solicitud.lower() in {"salir", "exit", "quit"}:
            print("Agente: Hasta luego.")
            break

        historial.append({"role": "user", "content": solicitud})
        resultado = agent.invoke({"messages": historial}, config=
{"recursion_limit": 15})

        structured: RespuestaAgente = resultado["structured_response"]
        historial = resultado["messages"]

        print(f"Agente: {structured.resumen}")
        print(f"  Acciones recomendadas:
{structured.acciones_recomendadas}")
        if structured.fuentes:
            print(f"  Fuentes: {' ', ' '.join(structured.fuentes)}")
        if structured.tools_usadas:
            print(f"  Herramientas usadas: {' ',
' '.join(structured.tools_usadas)}")
        print()
    except KeyboardInterrupt:
        print("\nAgente: Sesión finalizada por el usuario.")
        break
    except Exception as error:
        print("Agente: Ocurrió un error al procesar la solicitud.")
        print(f"Detalle técnico: {error}")

if __name__ == "__main__":
    if AGENT_MODEL == "anthropic" and not os.getenv("ANTHROPIC_API_KEY"):
        raise EnvironmentError(
            "No se encontró ANTHROPIC_API_KEY. Agregala al archivo .env, o usa
"

            "AGENT_MODEL=gemma-lmstudio para no depender de la API de pago."
        )
    iniciar_chat()

```